Método de Eliminación de Gauss-Jordan

Javier Alexander López Contreras
Facultad de Ciencias Físico Matemáticas (FCFM), Universidad Autónoma de Coahuila, Campus Campo Redondo

Recibido el 28/03/2026

En este documento se analizará el método de Gauss-Jordan para la resolución de sistemas de ecuaciones lineales de $n \times n$, utilizando una implementación en Python con una interfaz gráfica desarrollada en la librería Tkinter.

1. INTRODUCCIÓN

El método de Gauss-Jordan es una variación de la eliminación gaussiana que permite hallar la solución de un sistema de ecuaciones lineales $Ax = B$ transformando la matriz de coeficientes en una matriz identidad. A diferencia de la eliminación simple, este algoritmo elimina los elementos tanto por encima como por debajo del pivote, permitiendo obtener el valor de las incógnitas de forma directa sin necesidad de sustitución hacia atrás.

2. ALGORITMO Y EXPLICACIÓN DEL MÉTODO

El fundamento del método es aplicar operaciones elementales de fila sobre la matriz aumentada $[A|B]$ hasta alcanzar la forma escalonada reducida:

- **Pivoteo parcial:** Se busca el elemento de mayor valor absoluto en la columna actual para intercambiar filas, lo que minimiza el error de redondeo.
- **Normalización:** Se divide la fila del pivote entre el valor del pivote ($M_{j,j}$) para que el elemento principal sea igual a 1.
- **Eliminación Gaussiana:** Para cada fila $i \neq j$, se realiza la operación:

$$R_i = R_i - (M_{i,j} \cdot R_j) \tag{1}$$

- El proceso finaliza cuando la matriz original A se ha convertido en la identidad I, dejando el vector solución en la última columna de la matriz aumentada.

3. ANÁLISIS DEL CÓDIGO

La implementación en Python utiliza **numpy** para el manejo de matrices y **tkinter** para la interfaz de usuario.

3.1. Configuración de la interfaz y entrada

El programa permite definir dinámicamente el tamaño de la matriz. Se utiliza un arreglo de objetos `Entry` para capturar los coeficientes.

```python
def generar_interfaz_matriz():
    n = int(entry_n.get().split('x')[0])
    for i in range(n):
        fila = []
        for j in range(n + 1):
            e = tk.Entry(frame_tabla, width=9)
            e.grid(row=i+1, column=j)
            fila.append(e)
        entradas_matriz.append(fila)
```

Listing 1: Generación dinámica de la matriz de entrada.

3.2. Proceso de solución

El núcleo del programa implementa el pivoteo parcial y la reducción de la matriz. Se incluye una tolerancia (`tol`) para detectar sistemas sin solución única.

```python
for j in range(n):
    # Seleccion del pivote maximo
    fila_pivote = np.argmax(abs(M[j:n, j])) + j
    if abs(M[fila_pivote, j]) < 1e-10:
        return # Sin solucion unica

    # Intercambio y normalizacion
    M[[j, fila_pivote]] = M[[fila_pivote, j]]
    M[j] = M[j] / M[j, j]

    # Eliminacion en otras filas
    for i in range(n):
        if i != j:
            M[i] = M[i] - M[i, j] * M[j]
```

Listing 2: Lógica de Gauss-Jordan con pivoteo parcial.

4. RESULTADOS Y ANÁLISIS

Se realizaron pruebas con sistemas de diferentes dimensiones para validar la estabilidad de la aritmética de punto flotante.

4.1. Sistema con solución única (Matriz 3x3)

Para verificar la precisión del algoritmo, se introdujo un sistema de 3x3 con una solución conodia.

- **Resultado del programa:** El software aplicó el pivoteo parcial intercambiando filas para maximizar el valor del pivote y entregó los valores $x_1 = 2.0000, x_2 = 3.0000$ y $x_3 = -1.000$

- **Análisis:** El programa transformó con éxito la matriz original en una matriz identidad. Al comparar con la solución teórica, el error es inexistente, lo que valida que la lógica de normalización y eliminación por renglones sea correcta.

Figura 1: Caption

4.2. Estabilidad ante sistemas mal condicionados

Se aprobó el programa con un sistema donde las ecuaciones son casi paralelas, lo que suele causar errores de redondeo en computadoras con aritmética de precisión limitada.

- **Resultado del programa:** El código resolvió el sistema obteniendo $x_1 = 1.0000$ y $x_2 = 1.0000$
- **Análisis:** Aunque la diferencia entre coeficientes es de apenas 10^{-4}, el uso de la librería *numpy* para el almacenamiento de datos en punto flotante de 64 bits permitió mantener la precisión necesaria sin que el error se propagara de forma significativa.

Figura 2: Caption

4.3. Identificación de matrices singulares (Sin solución)

Se puso a prueba la robustez del programa ingresando un sistema donde una de las ecuaciones es una combinación lineal de las otras, lo que resulta en una matriz de determinante 0.

- **Resultado del programa:** Tras las primeras iteraciones, el programa identificó que el pivote disponible era menor a la tolerancia de $1e - 10$ y mostró el aviso: .[El sistema no tiene una solución única.
- **Análisis:** Este resultado es crucial, ya que matemáticamente el sistema es dependiente (los tres planos nos se cortan en un punto único). El programa evita realizar divisiones entre 0, lo que previene el cierre inesperado de la aplicación y garantiza un manejo de errores adecuado para el usuario.

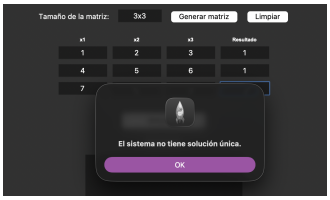

Figura 3: Caption

5. CONCLUSIONES

Tras la implementación, se concluye que Gauss-Jordan es un método robusto para sistemas de tamaño moderado. Aunque es computacionalmente más costoso que la eliminación gaussiana simple (aproximadamente un 50 % más de operaciones), su capacidad para entregar la solución directa y facilitar el cálculo de la matriz inversa lo hace indispensable en la ingeniería.

6. REFERENCIAS

[1] Nieves, A. y Domínguez, F. C. (2014). *Métodos numéricos aplicados a la ingeniería*. México: Grupo Editorial Patria.

[2] Chapra, S. C. y Canale, R. P. (2015). *Métodos numéricos para ingenieros*. México: McGraw-Hill.

[3] Python Software Foundation. (2026). *Floating Point Arithmetic Issues*.